

GORILATOR RPG · PR #101 · ISSUES #64-#73

The Feature Lab

Scenarios, bots & the event-module foundation — the workflow that turns a GitHub issue into a playable, self-verified feature.

10 issues one PR

124 unit tests 50 before

4/4 e2e green



The realm is now a sandbox — the tower defense is a plugin. **Every future feature ships with a lab.**

Build the tooling first



Before

- One hardcoded game loop — goblin waves besiege La Crypta, always
- Testing a feature = boot the full game and play it by hand
- `Math.random()` everywhere — a failing test is noise, not signal
- Persistent veterans inflate wave difficulty for newcomers



After (this PR)

- Open sandbox core + pluggable game loops (event modules)
- Every feature gets an isolated scenario map + scripted-bot self-test
- Seeded RNG — same seed, same run, trustworthy assertions
- Difficulty knobs on sliders; tuned values bake into `realm.json`

What shipped — ten issues, one stack



Seeded RNG service #70

reproducible rolls, per-cycle seed



Batched schema change #72

mana · hunger · gear · event HUD



La Crypta → plugin #73

tower defense extracted, core = sandbox



Scenario manifests #65

isolated test maps in scenarios/



Bot driver v1 #68

scripted players + headless harness



timeScale audit #67

16× accelerated simulation



Plugin API 1.1 #71

registerEventModule game loops



Difficulty knobs #64

relief valve for veteran waves



Scenario runner #66

pnpm scenario + ?scenario= link



Tweaks panel + skill #69

sliders, Bake, /feature-dev

Issue → playable feature: one pipeline

1

Read the issue

/feature-dev restates the loop; asks before coding if ambiguous

2

Implement

lowest tier that works: content → plugin → core

3

Author the lab

scenarios/<feature>.json stages ONLY that feature

4

Bot self-test

scripted players prove the loop headlessly, in vitest

5

Verification ladder

lint · typecheck · build · test · e2e · bench

6

Hand off a link

?scenario=<feature> — auto-joins single-player

7

You play & tune

Scenario tweaks panel: sliders apply live

8

Bake values

tuned numbers persist to realm.json

Definition of Done — all five: code · scenario manifest · bot self-test · tweak knobs · docs

Scenarios — an isolated test map per feature



One feature, nothing else

A JSON manifest in scenarios/ stages exactly what the feature needs: world entities, your loadout & stats, tuning overrides, bots.



Layers over your config

Merge order: defaults → realm.json → scenario → GORILATOR_TEST → live sliders. Last write wins.



Quiet by default

The realm event (waves + objective) and ambient spawners stay OFF unless the manifest turns them on.



Reproducible

"seed" pins every gameplay roll — same scenario + same seed = same run.

```
scenarios/hunger.json
{
  "name": "hunger",
  "seed": 1234,
  "world": {
    "groundItems": [{ "item": "banana",
      "x": 2, "z": 0, "count": 5 }]
  },
  "player": { "stats": { "hunger": 40 } },
  "systems": { "events": false,
    "spawners": false },
  "tuning": { "enemyMaxHp": 20 },
  "timeScale": 2,
  "bots": [{ "behavior": "eat_when_low" }]
}
```

Anatomy of a manifest

world

Stage the map: resources (tree | rock), groundItems, npcs (defId = model), enemies (goblin/dummy/defId).

player

Fresh joiners get this loadout (replaces starter items), whitelisted stats (incl. mana/hunger), and position.

systems

Toggles: events (the realm game loop) and spawners (ambient spawning). Both default OFF in a lab.

tuning

Any DevTuningKey. These ARE the declared tweak knobs — the panel pins exactly these keys.

timeScale

0–16. Accelerate the whole sim — grow, drain, cool down in seconds. Bot waits scale too.

bots

Scripted players spawned with the room: behavior, count, position, loadout, stats.

Shipped examples: scenarios/bot-arena.json (the template) · wave-siege.json · baseline.json — see scenarios/README.md

Running a lab — one command to playable

```
$ pnpm scenario bot-arena
[scenario] "bot-arena" selected
[scenario] ready-to-play:
  http://localhost:4901/?scenario=bot-arena

$ GORILATOR_SEED=68 pnpm dev
[rng] realm cycle seed = 68 (source: env)

$ curl :4902/api/status
{ "activeScenario": "bot-arena",
  "cycleSeed": { "seed": 68, ... } }
```



Env var is the source of truth

The runner sets GORILATOR_SCENARIO; on open dev servers ?scenario= alone also works for a fresh room.



No splash, no login

The ?scenario= link skips credentials and auto-joins single-player.



Wipes re-stage the lab

A realm reset rebuilds the staged world with the same pinned seed.

Per-worktree ports (.claude/launch.json) — labs in parallel worktrees never collide.

Bot driver — scripted players

Primitives (compose with seq / loop)

`moveTo · pickupNearest · attackNearest · eat`
`· waitFor · waitUntil · say`

Reserved for their future systems (fail loudly today):

`craft · equip · cast · plant`

Every primitive has a sim-time timeout — a stuck bot reports
`"failed"`, never hangs a test.

Builtin behaviors

wander

aggro

loot

eat_when_low

Bots are ordinary Players (bot-0, bot-1...) — movement, combat, stamina and pickups treat them like anyone. **registerBotBehavior** adds your own.



Two run modes. Headless: `createScenarioSim` runs the real systems in vitest — no sockets, no ports. Live: scenarios auto-spawn their bots[], or use Dev Mode → Bots / the `dev_bot` message (max 8).

The bot self-test, in vittest

```
packages/server/src/systems/bots/botArena.test.ts — the template  
  
const sim = createScenarioSim({  
  scenario: loadScenario("bot-arena")!,  
});  
  
sim.runUntil((state) =>  
  allEnemiesDead(state));  
  
expect(countItem(  
  sim.inventories.get(lootBot!),  
  "banana")).toBe(3);  
  
sim.dispose(); // ALWAYS (afterEach)
```



Real systems, real order

Composes the actual movement/combat/AI systems in GameRoom tick order at 50ms steps.



Assert state, not pixels

Inspect GameState + recorded broadcasts — the repo's DOM-not-canvas rule.



Determinism proof

Same scenario + same seed → byte-identical world evolution (it's a test in the template).

This test IS the “bot self-test” artifact of the Definition of Done.

Your part: play it, feel it, move sliders

1



Open the link

?scenario=<name> auto-joins; the server announces the scenario to your client

2



Open Dev Mode → Gameplay

a “Scenario tweaks — <name>” section sits pinned on top with the manifest's knobs

3



Tune live

sliders apply instantly via dev_tune — no restart, no config file

4



Bake values

one click writes the tuned numbers into realm.json's tuning block — they survive the session

Also in the panel

Bots section

spawn/clear scripted players while you watch

16× time bar

fast-forward the whole sim (and bot waits)

Numeric fallback

brand-new knobs get an input even before a slider exists

Bake is a Vite dev endpoint (/__scenario/bake) — it only ever touches realm.json's tuning block.

/feature-dev — the pipeline as an agent skill

```
> /feature-dev implement issue #77  
# hunger, food, cooking & farming
```

The skill walks the whole Definition of Done: it reads the issue, asks 2-3 pointed questions if the spec is ambiguous, implements at the lowest tier, authors the scenario, writes the bot self-test, runs the ladder — and ends by handing you a ready-to-play link.

“The human's job collapses to the part humans are for: play it, feel it, tune the sliders, say yes or no.”

What the skill enforces

- Clarify before coding — ambiguous numbers or interactions become questions, not guesses
- Lowest tier that works: content JSON → plugin → core
- New systems: perf span + pure (state, dt) + seeded rng() only
- Scenario cranked so the full loop plays out in under a minute
- Bot self-test from the botArena template before any human plays
- Full verification ladder before the hand-off

PART TWO

Under the hood

The engine changes every future feature stands on



seeded RNG



timeScale 16x



schema batch



plugin API 1.1

Seeded RNG — the dice are now a service

```
// gameplay code
const roll = rng(state, "drops");
if (roll() < rule.probability) ...

// tests — never spy Math.random again
installFixedRng(state, 0.5);

// pin a whole run
GORILATOR_SEED=68 pnpm dev
```

⚠ **Guard test:** `rng.test.ts` greps the server source — any new `Math.random()` in game code FAILS the suite. Use `rng(state, stream)`.



Independent streams

combat · drops · spawns · ai · world · bots · misc —
adding a roll in one never shifts another's
sequence.



One seed per realm cycle

scenario seed → GORILATOR_SEED → random.
Logged at boot, reported at `/api/status`.



Injected, not ambient

Keyed by the `GameState` object — pure (state, dt)
signatures unchanged.

timeScale — fast-forward the sim

16x

new cap (TIME_SCALE_MAX) — the Dev
Mode time bar now goes $\text{II} \cdot \frac{1}{2} \cdot 1 \cdot 2 \cdot 4 \cdot$
 $8 \cdot 16$

Scales with time

- every (state, dt) system
- regrow & respawn timers
- house regen, buffs, throws
- event-module wave clocks
- bot waitFor / waitUntil

Wall-clock by design

- game-over intermission
- Dev ghost-cam while paused
- perf tracking & benchmarks
- content watchFile reloads
- Nostr challenge expiry

Equivalence-tested: N ticks at $1x \equiv N/2$ at $2x$; $0x$ freezes everything. Above $16x$ a 20 Hz tick starts tunnelling through collision — sub-stepping is the documented follow-up.

Plugin API 1.1 — game loops as plugins

```
plugins/<name>/src/server.ts — compiles against @rpg/shared ONLY
ctx.registerEventModule({
  id: "my-event",
  onStart(ev) {
    ev.world.spawnStructure({ kind: "house" });
  },
  onTick(ev, dt) {
    ev.world.spawnEnemy({ ... });
    ev.setEventHud({ timerMs, progress });
    if (lost) ev.endEvent({ result: "defeat" });
  },
  onCommand(ev, cmd) { /* force_next_wave */ },
});
```



Host-owned world mutators

spawnEnemy · spawnStructure · giveItem · grantXp · broadcast — plus ev.tuning(key), ev.rng(stream), ev.customWave(n).



New lifecycle events

realm:start · entity:damaged · event:start/end · objective:complete — fired at host chokepoints.



Safe by construction

Perf-spanned as event:<id>; a throwing module is logged, never kills the 20 Hz tick. Additive: ^1.0.0 plugins still load.

La Crypta is now a plugin

Moved to the plugin

- wave scheduler + clock
- La Crypta spawn + regen
- home-fall → defeat → wipe
- event HUD sync

Stays in core

- goblin AI + all brains
- combat & damage formula
- schema, spawners, leaderboards
- waves.json content pipeline

```
realm.json
"events": {
  "enabled": true,
  "autoStart": true,
  "module":
    "la-crypta-defense",
  "config": {
    "difficultyMult": 1
  }
}
```



Proof it changed nothing: the characterization suites (wave pacing, sizes, levels, spread, regen, defeat flow) were written against the OLD core first — and pass unchanged against the plugin. Disable it (events.enabled: false or GORILATOR_TEST=1) → open sandbox: no house, no waves, no wipe.

Difficulty knobs — the relief valve

Progression persists across wipes — so returning veterans inflate wave size and enemy levels for everyone. These knobs put that curve on sliders.

difficultySizeScale

0.1 – 5

×N on wave size (before the waveSizeMax cap)

difficultyLevelScale

0 – 5

×N on the per-wave level escalation — 0 = never escalate

difficultyLevelCap

0 – 200

hard ceiling on spawned enemy levels — 0 = uncapped

Tunable from

- realm.json tuning
- scenario manifests
- Dev Mode sliders
- `events.config.difficultyMult` (per-event ×)

At the defaults (1 / 1 / 0) every formula reduces exactly to legacy behavior — characterization-pinned. Try it staged: `pnpm scenario wave-siege`.

One batched schema change

```
packages/shared/src/schema — @type synced fields
// GameState — the event HUD
eventId  eventLabel
eventTimerMs  eventProgress

// Player — survival & visible gear
mana  maxMana
hunger  maxHunger
gearWeapon  gearHead
gearChest  gearBoots
```

Equipment (#75), survival (#77) and abilities (#78) write these fields later — no further schema release needed.



Hard reload required

Any @type change needs pnpm build:shared + a client HARD reload — HMR is not enough. That cost is WHY changes are batched.



The rule going forward

No new @type fields outside a planned batch. Per-player state (quests, inventory) uses server Maps + owner-only client.send().



Why it matters for perf

Schema syncs every frame to every client — keeping it lean is browser-game discipline.

Tips & gotchas — the field guide



Never `Math.random()`

the `rng.test.ts` source guard fails the suite — use `rng(state, "stream")`



Manifests beat tuning

`entity-features.json` per-entity stats override scenario tuning (goblin hp 45 beats `enemyMaxHp 20`) — stage dummies for exact stats



Silence the spawners

set `systems.spawners`: false or ambient goblins wander into your lab



Always `sim.dispose()`

`devTuning/realmEvents` are module-global — undisposed sims leak tuning across test files



Hard reload after schema

stale clients desync silently against a new shared build



`GORILATOR_TEST=1` = events off

the old `wave-size-0` hack is gone; scenario bots still spawn in test mode

WHAT'S NEXT

Every remaining issue is now a /feature-dev run

```
cheat sheet
# stage & play any lab
pnpm scenario bot-arena
pnpm scenario wave-siege

# pin the dice / inspect the run
GORILATOR_SEED=68 pnpm dev
curl :4902/api/status # scenario + seed

# build the next feature
/feature-dev implement issue #77
```

The queue (Phase 3)

```
#75 equipment & wearables
#76 crafting + stations
#77 hunger, food & farming
#78 abilities & action bar
#79 threat & trinity combat
#80 parties · #81 quests
#74 default sandbox loop
```

Each ships with its own scenario, bots and knobs — **you just play the link and say yes or no.**

github.com/agustinkassis/gorilator-rpg/pull/101 · [docs/feature-lab.md](#) · [scenarios/README.md](#)